

Projet SIO-Shoes : site « e-commerce » avec Symfony

Partie 5 : menu et rôles des utilisateurs

1. Gestion des droits d'accès pour les catégories.

Ouvrez le fichier `config/packages/security.yaml` et modifiez l'`access_control` comme ceci :

```
# Easy way to control access for large sections of your site
# Note: Only the *first* access control that matches will be used
access_control:
- { path: ^/admin, roles: ROLE_ADMIN }
  - { path: ^/editor, roles: ROLE_EDITOR }
  # - { path: ^/profile, roles: ROLE_USER }
```

Allez dans la table `user` de la base de données et modifiez le champ « rôle » d'un utilisateur :
["ROLE_ADMIN", "ROLE_EDITOR", "ROLE_USER"]

Allez dans « `src/Controller/CategoryController.php` » et modifiez le chemin des routes pour qu'elles soient réservées au rôle « `ROLE_ADMIN` » :

```
#[Route('/admin/category', name: 'app_category')]
#[Route('/admin/category/new', name: 'app_category_new')]
#[Route('/admin/category/{id}/update', name: 'app_category_update')]
#[Route('/admin/category/{id}/delete', name: 'app_category_delete')]
```

Testez les nouvelles routes sans être connecté puis en étant connecté avec un utilisateur ayant le rôle « `ROLE_ADMIN` ».

2. Ajout d'un menu de navigation responsive (nav bar).

Allez sur le site <https://getbootstrap.com/> puis copiez le script qui permet d'inclure Bootstrap JavaScript. Par exemple :

```
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.6/dist/js/bootstrap.
bundle.min.js" integrity="sha384-
j1CDi7MgGQ12Z7Qab0qlWQ/Qqz24Gc6BM0thvEMVjHnFYGF0rmFCozFSxQBxwHKO"
crossorigin="anonymous"></script>
```

Collez ensuite le script dans l'entête (Head) du fichier « `templates/base.html.twig` ».

3. Récupération du code de la « Navbar » .

Sur le site <https://getbootstrap.com/> , allez dans « Docs » puis « Navbar ». Allez dans « Scrolling » puis copiez le code. Créez le fichier « `templates/layouts/nav.html.twig` » et collez le code. Modifiez les trois premières lignes comme ci-dessous :

```
<!-- Lignes à modifier -->
<nav class="navbar navbar-expand-lg bg-body-tertiary" data-bs-theme="dark">
  <div class="container">
    <a class="navbar-brand" href="#">My shop</a>
```

```

<!-- Fin modifications -->
<button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-
bs-target="#navbarScroll" aria-controls="navbarScroll" aria-expanded="false"
aria-label="Toggle navigation">
<span class="navbar-toggler-icon"></span>
</button>
<div class="collapse navbar-collapse" id="navbarScroll">
<ul class="navbar-nav me-auto my-2 my-lg-0 navbar-nav-scroll" style="--bs-
scroll-height: 100px;">
<li class="nav-item">
    <a class="nav-link active" aria-current="page" href="#">Home</a>
</li>
<li class="nav-item">
    <a class="nav-link" href="#">Link</a>
</li>
<li class="nav-item dropdown">
    <a class="nav-link dropdown-toggle" href="#" role="button" data-bs-
toggle="dropdown" aria-expanded="false">Link</a>
    <ul class="dropdown-menu">
        <li><a class="dropdown-item" href="#">Action</a></li>
        <li><a class="dropdown-item" href="#">Another action</a></li>
        <li><hr class="dropdown-divider"></li>
        <li><a class="dropdown-item" href="#">Something else here</a></li>
    </ul>
</li>
<li class="nav-item">
    <a class="nav-link disabled" aria-disabled="true">Link</a>
</li>
</ul>
<form class="d-flex" role="search">
    <input class="form-control me-2" type="search" placeholder="Search"
aria-label="Search"/>
    <button class="btn btn-outline-success" type="submit">Search</button>
</form>
</div>
</div>
</nav>

```

4. Ajout de la gestion des catégories dans le menu.

Allez dans le fichier « templates\layouts\nav.html.twig » et modifiez le menu dropdown :

```

{% if is_granted("ROLE_ADMIN") %}
<li class="nav-item dropdown">
    <a class="nav-link dropdown-toggle" href="#" role="button"
data-bs-toggle="dropdown" aria-expanded="false">Admin</a>
    <ul class="dropdown-menu">

```

```

        <li><a class="dropdown-item" href="{{ path('app_category') }}">
        Catégories</a></li>
        <li><a class="dropdown-item" href="#">Another action</a></li>
        <li><hr class="dropdown-divider"></li>
        <li><a class="dropdown-item" href="#">Something else here</a></li>
    </ul>
</li>
{% endif %}

```

Remarque : « `if is_granted("ROLE_ADMIN")` » permet de n'afficher le drop-down menu qui si l'utilisateur connecté possède le rôle « `ROLE_ADMIN` ».

5. Afficher le menu dans toutes les pages

Allez dans le fichier « `templates\base.html.twig` » et ajoutez le code suivant pour appeler votre « navbar » dans la page de base :

```

<body>
    {% block nav %}
        {% include 'layouts/nav.html.twig' %}
    {% endblock %}

    {% block body %}{% endblock %}
</body>

```

6. Ajout de la gestion des utilisateurs.

Créez le contrôleur pour les utilisateurs :

```
symfony console make:controller UserController
```

Ouvrez ensuite le fichier « `src\Controller\UserController.php` » pour modifier la route « `/user` » :

```

#[Route('/admin/user', name: 'app_user')]
public function index(UserRepository $userRepository): Response
{
    $users = $userRepository->findAll();
    return $this->render('user/index.html.twig', [
        'users' => $users
    ]);
}

```

Remarque : la gestion des utilisateurs doit être réservée aux administrateurs.

N'oubliez pas d'importer les classes suivantes :

```

use App\Entity\User;
use App\Repository\UserRepository;
use Doctrine\ORM\EntityManagerInterface;

```

7. Modification de la vue pour afficher les utilisateurs dans un tableau.

Allez dans le fichier « `templates/user/index.html.twig` » et modifiez le code comme ci-dessous :

```

{% extends 'base.html.twig' %}

{% block title %}Utilisateurs{% endblock %}

```

```

{% block body %}
    <div class="container">
        <br>
        <h1>Utilisateurs</h1>
        <br>
        {% include 'layouts/_flash_message.html.twig' %}
        <table class="table">
            <tr>
                <th>id</th>
                <th>email</th>
                <th>nom</th>
                <th>prénom</th>
                <th>rôles</th>
                <th>actions</th>
            </tr>
            {% for user in users %}
            <tr>
                <td>{{ user.id }}</td>
                <td>{{ user.email }}</td>
                <td>{{ user.firstName }}</td>
                <td>{{ user.lastName }}</td>
                <td>
                    {% if ("ROLE_ADMIN" in user.roles) %}
                        <span class="text-primary">administrateur</span>
                    {% elseif ("ROLE_EDITOR" in user.roles) %}
                        <span class="text-success">editeur</span>
                    {% else %}
                        <span class="text-secondary">client</span>
                    {% endif %}
                </td>
            </tr>
            {% else %}
            <p>Aucun utilisateur à afficher</p>
            {% endfor %}
        </table>
    </div>
{% endblock %}

```

8. Ajouter les routes pour modifier les rôles des utilisateurs.

Allez dans le fichier « src\Controller\UserController.php » et ajoutez les deux routes suivantes :

```

#[Route('/admin/user/{id}/add/editor', name: 'app_user_add_editor_role')]
public function editorRoleAdd(User $user, EntityManagerInterface
$entityManager):Response

```

```

{
    $user->setRoles(["ROLE_EDITOR", "ROLE_USER"]);
    $entityManager->flush();

    $this->addFlash('success','Le rôle éditeur a été ajouté à l\'utilisateur');
    return $this->redirectToRoute('app_user');
}

#[Route('/admin/user/{id}/remove/editor', name: 'app_user_remove_editor_role')]
public function editorRoleRemove(User $user, EntityManagerInterface
$entityManager):Response
{
    $user->setRoles([]);
    $entityManager->flush();

    $this->addFlash('danger','Le rôle éditeur a été retiré à l\'utilisateur');
    return $this->redirectToRoute('app_user');
}

```

9. Ajoutez les liens vers le changement de rôle dans la vue.

Allez dans le fichier « templates/user/index.html.twig » et ajoutez le code suivant à la fin du tableau :

```

<td>
{% if ("ROLE_ADMIN" in user.roles) == false %}
    {% if ("ROLE_EDITOR" in user.roles) == false %}
        <a onclick="return confirm('Voulez-vous vraiment affecter le rôle
        éditeur à l\'utilisateur ?')" class="btn btn-outline-primary" href="{{
        path('app_user_add_editor_role',{'id':user.id}) }}">Ajouter le rôle
        Editeur</a>
    {% else %}
        <a onclick="return confirm('Voulez-vous vraiment retirer le rôle
        éditeur à l\'utilisateur ?')" class="btn btn-danger" href="{{
        path('app_user_remove_editor_role',{'id':user.id}) }}">Retirer le rôle
        Editeur</a>
    {% endif %}
{% endif %}
</td>
</tr>

```

10. Ajouter la route pour supprimer un utilisateur.

Allez dans le fichier « src\Controller\UserController.php » et ajoutez le code suivant :

```

#[Route('/admin/user/{id}/delete', name: 'app_user_delete')]
public function editorRoleDelete(User $user, EntityManagerInterface
$entityManager):Response
{

```

```

$entityManager->remove($user);
$entityManager->flush();

$this->addFlash('danger','L\'utilisateur a été supprimé');
return $this->redirectToRoute('app_user');
}

```

11. Ajoutez le lien pour supprimer un utilisateur dans la vue.

Allez dans le fichier « templates/user/index.html.twig » et modifiez le code à la fin du tableau :

```

<td>
{% if ("ROLE_ADMIN" in user.roles) == false %}
    {% if ("ROLE_EDITOR" in user.roles) == false %}
        <a onclick="return confirm('Voulez-vous vraiment affecter le rôle
        éditeur à l\'utilisateur ?')" class="btn btn-outline-primary" href="{{
        path('app_user_add_editor_role',{'id':user.id}) }}">Ajouter le rôle
        Editeur</a>
    {% else %}
        <a onclick="return confirm('Voulez-vous vraiment retirer le rôle éditeur
        à l\'utilisateur ?')" class="btn btn btn-danger" href="{{
        path('app_user_remove_editor_role',{'id':user.id}) }}">Retirer le rôle
        Editeur</a>
    {% endif %}
{% endif %}
<a onclick="return confirm('Voulez-vous vraiment supprimer cet utilisateur ?')"
class="btn btn-danger" href="{{ path('app_user_delete',{'id':user.id})
}}">Supprimer</a>
</td>
</tr>

```